

INTRO TO WHITEBOX PENTESTING

CHEAT SHEET

The Whitebox Pentesting Process

Order	Step	Description
1.	Code Review	General review of the code to understand its functionality and shortlist potentially vulnerable functions
2.	Local Testing	Testing/Debugging the code locally to test our findings and identify vulnerabilities
3.	Proof of Concept	Writing an exploit to prove the exploitability of the target automatically
4.	Patching & Remediation	Patching the vulnerability and all of its sources/causes

Examples of Code Review Techniques

1. **Select functions based on the application design.**
2. **Select functions and files through search.**
3. **Select functions through the use of the application.**

Exploit Scripting Language Guide

Use Case	Recommended Language	Reason
Attack is on a network application (including web applications)	Python	It works similarly on most operating systems
Attacking a client-side function (e.g. a CSRF attack)	JavaScript	It is the only script executed by browsers
Web chain including a client-side attack	Python & JavaScript	We prepare a JavaScript payload for the client-side part. Then, use it with a Python script to trigger the exploit and carry on the rest of the back-end attacks.
Binary exploitation	Python	Python has good libraries for debugging and exploiting binaries, while C/C++ may be used to develop a binary exploit.
Targeting an operating system	Bash or PowerShell/CMD	Whatever pre-installed scripting language on that operating system
Thick client or some advanced types of exploitation	The application's programming language	This would enable us to reuse code/functions to generate some payloads, which would save us a lot of time (vs re-scripting all of the logic in Python)

Code/Command Injection Functions

JavaScript 'NodeJS'	Python	PHP	C/C++	C#	Java
<code>eval</code>	<code>eval</code>	<code>eval</code>	<code>execlp</code>		
Function	<code>exec</code>	<code>exec</code>	<code>execvp</code>		
<code>setInterval</code>	<code>subprocess.open</code>	<code>proc_open</code>	<code>ShellExecute</code>		
<code>setTimeout</code>	<code>subprocess.run</code>	<code>popen</code>			
<code>constructor.constructor</code>	<code>os.system</code>	<code>shell_exec</code>			
<code>child_process.exec</code>	<code>os.popen</code>	<code>passthru</code>	<code>system</code>	<code>System.Diagnostics.Process.Start</code>	<code>Runtime.getRuntime().exec</code>
<code>child_process.spawn</code>		<code>system</code>	<code>popen</code>		

Payload Development Rules

1. Comment out the rest of the code
2. Ensure quotes/parentheses/curly braces are even
3. Maintain a working function without syntax errors

Methods for Obtaining Command Output

1. Log output to console "for local testing"
2. Use a reverse shell
3. Use DNS exfiltration (or ping exfiltration)
4. Store the output in the database
5. Write the output to a file, then access that file
6. Inject the output into the HTTP response
7. Use sleep timers or boolean output to read the content